

An Introduction to MolAICal Scripts, Installation within the Linux MolAICal Container, and Instructions for Running the Full Linux Version of MolAICal on Windows and macOS

Qifeng Bai

Email: molaical@yeah.net

Homepage: <https://molaical.github.io> or <https://molaical.gitlab.io>

School of Basic Medical Sciences

Lanzhou University

Lanzhou, Gansu 730000, P. R. China

Content

1. Introduction.....	2
2. Materials	3
3. Solving the library file dependency issue of the Linux version of MolAICal	3
3.1 Install external programs inside the MolAICal container (Recommendation)	3
3.2 Share the host machine's external software along with the dependent library files.....	5
4 MolAICal batch Script.....	6
4.1 Principle of MolAICal Script.....	6
4.2 Arguments of MoAICal.....	7
5 Run the full version of Linux MolAICal on Windows and macOS operating systems	9
6 Linux Version of MolAICal: Architecture and Acceleration	10
Appendix 1.....	12

1. Introduction

MolAICal can execute programs line by line according to its command line, which facilitates the development and efficient operation of AI agents. Meanwhile, the Linux version of MolAICal will later run in container form, making it more universal by not relying on library files. On Windows, it might be possible to run MolAICal directly on Windows 10 or later using the Windows Subsystem for Linux (WSL), or to run MolAICal by deploying its image via Docker Desktop for Windows - x86_64 (<https://docs.docker.com/desktop/setup/install/windows-install>); Or Docker Desktop on Mac (<https://docs.docker.com/desktop/setup/install/mac-install>) is used for MacOS. Here, we discuss some special topics such as MolAICal batch scripts and container installation programs.

2. Materials

2.1. Software requirement

1) MolAICal: <https://molaical.github.io> or <https://molaical.gitlab.io>

2) NAMD (CPU version): <https://www.ks.uiuc.edu/Research/namd/>

(Notice: This tutorial can be run by NAMD 2.x, 3.x, or a higher version. For example, the command “**namd3**” substitutes for the command “**namd2**” in this tutorial if you use NAMD 3.x version. For a higher version of NAMD, users can use a similar replacement way as the previous example.)

3) VMD: <https://www.ks.uiuc.edu/Research/vmd>

2.2. Example files

1) All the necessary tutorial files are downloaded from:

https://github.com/MolAICal/tutorials/tree/master/030-MolAICal-special_topics

3. Solving the library file dependency issue of the Linux version of MolAICal

Containers like Docker offer a key advantage: They enable the sharing of folders, files, and library files with the host machine through mounting, which can maximally resolve dependency issues such as library file requirements. For the Linux version of MolAICal deployed in containers, if external programs like VMD, NAMD, or UCSF Chimera need to be invoked, there are two approaches:

1) Directly install external programs (e.g., VMD, NAMD, UCSF Chimera) **inside** the MolAICal container and configure internal files (**Recommendation**) →3.1.

2) Share the host machine's external software (e.g., VMD, NAMD, UCSF Chimera) **along with dependent library files** via mounting →3.2.

3.1 Install external programs inside the MolAICal container (Recommendation)

a) First of all, copy the files to the MolAICal container.

```
***** 1. Go to the container file system, it will enter the "/root" *****
```

```
#> molaical.exe -eset shell in
```

```
***** 2. The first part of 'cp' is from the local machine, and the second part is in the container; *****
```

```
***** The VMD and NAMD packages can be moved into the container by the command below *****
```

```
#> cp /home/user/<local machine file> /root/soft
```

```
***** 3. Exit container file system *****
```

```
#> exit
```

b) Secondly, enter the container virtual environment of MolaICal:

```
#> molaical.exe -eset sys run molaical
```

Note: 'molaical' is the container name.

c) Installing software in the container virtual environment of MolaICal is the same as installing it on the host local machine. Here, we take the installation of VMD and NAMD as examples:

1) For NAMD:

Extract the NAMD files, assuming the extracted folder is named namdcpu. Then specify its path to MolaICal using the following command:

```
#> molaical.exe -call set -n NAMD -p "/root/soft/namdcpu/namd3"
```

Note: Please **replace** the above paths for VMD and NAMD with the **actual paths** on the users' system. The string "VMD" and "NAMD" (case insensitive) after "-n" is fixed for the paths of VMD and NAMD. To ensure the reproducibility of MM/GBSA results, it is recommended to use the CPU version of NAMD, as the "seed" parameter appears to be ineffective for reproducibility in the CUDA version of NAMD.

2) For VMD:

Following the steps below:

✧ Extract the VMD files:

```
#> tar -xzf vmd-xxx.tar.gz
```

Note: Please **replace** the above paths for VMD and NAMD with the **actual paths** on the users' system.

✧ Modify the install path in a file named "configure" in the decompressed folder of VMD:

The default: \$install_bin_dir="/usr/local/bin"; modify it to →

\$install_bin_dir="/root/soft/vmd193/bin";

The default: \$install_library_dir="/usr/local/lib/\$install_name"; modify it to →

\$install_library_dir="/root/soft/vmd193/lib/\$install_name";

✧ Install VMD:

```
#> cd vmd-xx
```

```
#> ./configure LINUXAMD64
```

```
#> cd src
```

```
#> make install
```

Note: Please run ./configure, and select the right types for the used computers. Here, it is "LINUXAMD64".

✧ Then specify its path to MolAICal using the following command:

```
#> molaical.exe -call set -n VMD -p "/root/soft/vmd193/bin/vmd"
```

So far, all the installations and configurations of NAMD and VMD are complete in the MolAICal container.

✧ Remember to use the "exit" command to quit the MolAICal virtual environment and return to the local computer for computation (primarily to avoid file-copying steps; **execution within the container is also possible but requires copying data from the local computer to the MolAICal container**).

```
#> exit
```

3) Save and load the modified container (Option)

To preserve changes made within a container and avoid reinstalling software later, since all data will be lost if the container is deleted.

◆ The first step is to obtain the container ID and name:

```
#> molaical.exe -eset sys ps
```

◆ Secondly, clone this container as follows:

```
#> molaical.exe -eset sys export --clone -o molaicalv2.tar molaical
```

Note: 'molaical' is the container name. It can also be the container ID. If an error is reported, it may be due to permission issues with the mounted file system, which can be ignored.

◆ Thirdly, import the cloned container **if needing**:

```
#> molaical.exe -eset sys import --clone --name molaicalv2 molaicalv2.tar
```

◆ Now, change the loaded **new container** name to the default container name "**molaical**", the **previous container** is renamed to "**bakmolaical**", the commands are as follows:

```
#> molaical.exe -eset sys rename molaical bakmolaical
#> molaical.exe -eset sys rename molaicalv2 molaical
```

Notice: Please note that a **container (dynamic)** is generated from an **image (static)**. Operations such as software installation must be performed within the dynamic container; if cloning this container, restoring it will still be based on the original underlying image.

For more information, please see the manual of MolAICal or run "**molaical.exe -eset sys --help**" to get more help details.

3.2 Share the host machine's external software along with the dependent library files

If the local computers have installed VMD, NAMD, and so on, **users do not want to install them**

in the container virtual environment of MolAICal. It should share the locally related necessary files, libraries, and others for the MolAICal container.

Here, we take the Ubuntu system as an example. For other systems, locate the corresponding library files' position first—note that library file locations vary across different systems. If the previous paths are cluttered, clear them by using the command:

```
#> molaical.exe -eset clean
#> molaical.exe -call set -o clean
```

Users can see the external path of MolAICal by:

```
#> molaical.exe -call set -o all
```

A. Tell the local paths of NAMD and VMD to the MolAICal container by:

```
#> molaical.exe -call set -n VMD -p "/home/feng/soft/vmd193/bin/vmd"
#> molaical.exe -call set -n NAMD -p "/home/feng/soft/namdcpu/namd3"
```

B. Tell the necessary libraries on the local machine to the MolAICal container by:

```
#> molaical.exe -eset lib /usr/lib/x86_64-linux-gnu
#> molaical.exe -eset bin /usr/lib/x86_64-linux-gnu
#> molaical.exe -eset mount /usr/lib/x86_64-linux-gnu
#> molaical.exe -eset lib /usr/lib
#> molaical.exe -eset bin /usr/lib
#> molaical.exe -eset mount /usr/lib
```

Then, go to the material folder "020-mmpbsa", and test it:

```
#> molaical.exe -mmpbgbsa -f mmpbsa_apbs.vmd
```

4 MolAICal batch Script

MolAICal will run the script (commands' list ruuning in Windows DOS or Linux command console) if the value of "-c" is "script".

4.1 Principle of MolAICal Script

Figure 4.1.1 provides a structured view of the MolAICal script execution process. Here's a detailed, professional description:

- ◆ MolAICal serves as the primary module, initiating the execution of the main Script.
- ◆ The Script contains sequential commands, such as Command 1, which triggers Script 1.
 - Command 1 executes first, followed by Script 1.
 - After Script 1 is executed, it may loop back to execute further commands, including

Command 1 again or other commands like Command n_i .

- ◆ Command n_i can be a different operation within the script. It also leads to the execution of Script n_i .
- ◆ This setup demonstrates a recursive, modular process, where commands and scripts interact and trigger subsequent operations, supporting a dynamic execution flow.

The blue arrows indicate direct connections between commands and their respective scripts, and the horizontal blue arrow illustrates the flow between command executions.

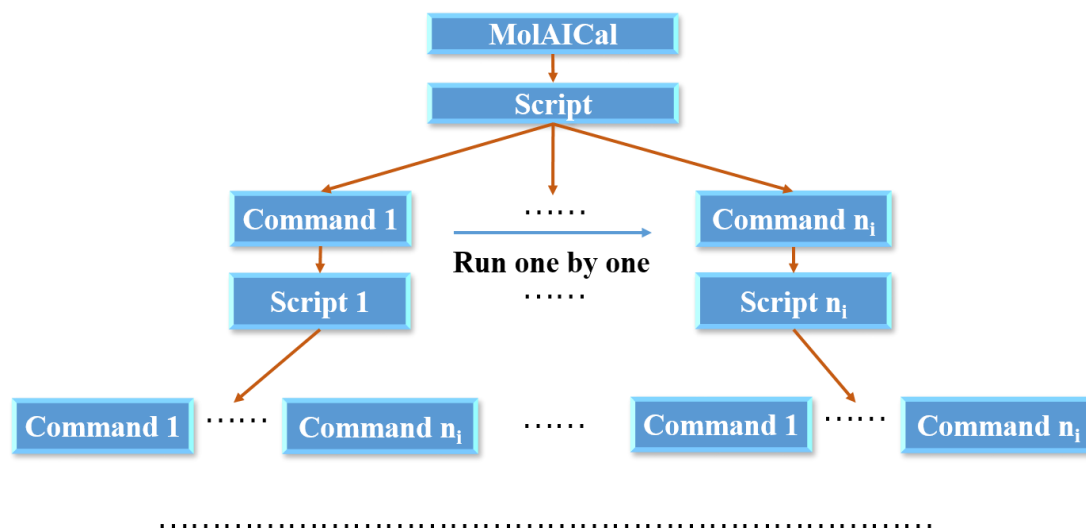


Figure 4.1.1 The framework of the MoAICal script

4.2 Arguments of MoAICal

The strings after '-i' are the parameter list that will be split by space. The split strings will replace the strings \$molargs1, \$molargs2, \$molargs3, etc. in the script file. The number of the string \$molargs1, \$molargs2, \$molargs3, etc., is the same as the length of the split strings after '-i'. Some special characters, such as the comment "#" and a string space " ", are shown as below:

- ◆ If a \$molargs $[x]_{x=1,2,...}$ is the string "^*^", it represents comments (e.g., #).
- ◆ If a \$molargs $[x]_{x=1,2,...}$ is the string "^^^", it represents a string space " ".

The examples are as follows:

1. No modification run in the file 'runscript'

```
#> molaical.exe -call run -c script -n runscript
```

2. Modify \$molargs1 in the file 'runscript' to complex.dcd

```
#> molaical.exe -call run -c script -n runscript -i complex.dcd
```

3. Modify \$molargs1 and \$molargs2 in the file 'runscript' to complex.dcd and complex.psf

```
#> molaical.exe -call run -c script -n runscript -i complex.dcd complex.psf
```

The "runscript" can be (just an example):

```
# 1. fix autopsf
molaical.exe -call run -c fixpsf

# 2. generate psf
molaical.exe -call run -c vmdargs -i -pdb $molargs1 -dispdev text -args autopsf autopsf -mol 0 -prefix protein
molaical.exe -call run -c vmdargs -i -pdb $molargs2 -dispdev text -args autopsf autopsf -mol 0 -top $molargs3 -
prefix ligand

# 3. merge file
molaical.exe -call run -c merge -i -dispdev text -args -first ligand_formatted_autopsf.psf
ligand_formatted_autopsf.pdb -second protein_formatted_autopsf.psf protein_formatted_autopsf.pdb -output
complex_final

# 4. default to run 20 ps minimization and 40 ps MD simulation
molaical.exe -call run -c runnamd -i -dispdev text -args -s complex_final.psf -c complex_final.pdb -nf
md_configure.conf -output_prefix com_md

# 5. Cut last 10 steps for MM/GBSA calculation
molaical.exe -call run -c catdcd -i -dispdev text -args -out result.dcd -first 51 -last 60 -stride 1 com_md.dcd

# 6. Calculate MM/GBSA
molaical.exe -call run -c mmpbgbsa -i -dispdev text -args -s complex_final.psf -c complex_final.pdb -t result.dcd
-cs "segname,AP1,CO1" -rs "segname,AP1" -ls "segname,CO1" -pb 2 -ff $molargs3

# 7. delete file with command (in windows: del xxx, in linux: rm xxx)
rm *md.* *run.* *formatted* result.dcd *final.* *_md* *_wb*
```

For the runnable example, please go to the example material "021-single_mmpbgbsa" for running a case of the MolaICal script.

5 Run the full version of Linux MolAICal on Windows and macOS operating systems

Download the Docker Desktop:

- ✧ **Install Docker Desktop on Mac:** <https://docs.docker.com/desktop/setup/install/mac-install>
- ✧ **Install Docker Desktop on Windows:**
<https://docs.docker.com/desktop/setup/install/windows-install>

1) Installing Docker Desktop is very easy: just click the downloaded executable file, Docker Desktop, and follow the prompts to complete the installation successfully.

2) Since the Linux version of MolAICal currently uses an independent udocker image, installing Docker Desktop on Windows and macOS operating systems allows running the full version of Linux MolAICal. The common steps running on **Windows DOS** or **macOS Terminal** as follows:

a) pull Ubuntu 20.04

```
#> docker pull ubuntu:20.04
```

Note: If you are unable to connect to Docker's image registry from your location, you will encounter the error:

Error response from daemon: Get "https://registry-1.docker.io/v2/": net/http: request canceled while waiting for connection (Client.Timeout exceeded while awaiting headers).

In this case, users need to **update** the image registry source according to the method below (see **Appendix 1 in this tutorial**). If there are no issues, please omit Appendix 1.

b) Start a container for running MolAICal based on this

```
#> docker run -itd --name molaical -w /root ubuntu:20.04 /bin/bash
#> docker exec -it molaical /bin/bash
#> apt update
#> apt install make -y
#> apt install vim -y
```

Note: if needing to install programs, "**apt update**" should be inputted and run in the terminal.

c) Copy MolAICal and experimental files into the Docker

```
#> docker cp <Your path>\MolAICal-linux64 molaical:/root/soft
```

Until now, MolAICal and materials are copied into the directory /root/ in the container named "molaicalvx".

d) Go into the Docker environment, install MolAICal, and run the case of MolAICal

```

***** Go to Windows or MacOS Docker container *****
#> docker exec -it molaical /bin/bash
***** Go to molaical container if wanting to install the external program *****
#> molaical.exe -cset sys run molaical
#> exit
#> mkdir workdir
#> cd /root/workdir
***** A peptide generation case *****
#> molaical.exe -call run -c pepgen -i 1 8 -n 2 -3d -nc -o linear_seq.txt -dir linear

```

6 Linux Version of MolAICal: Architecture and Acceleration

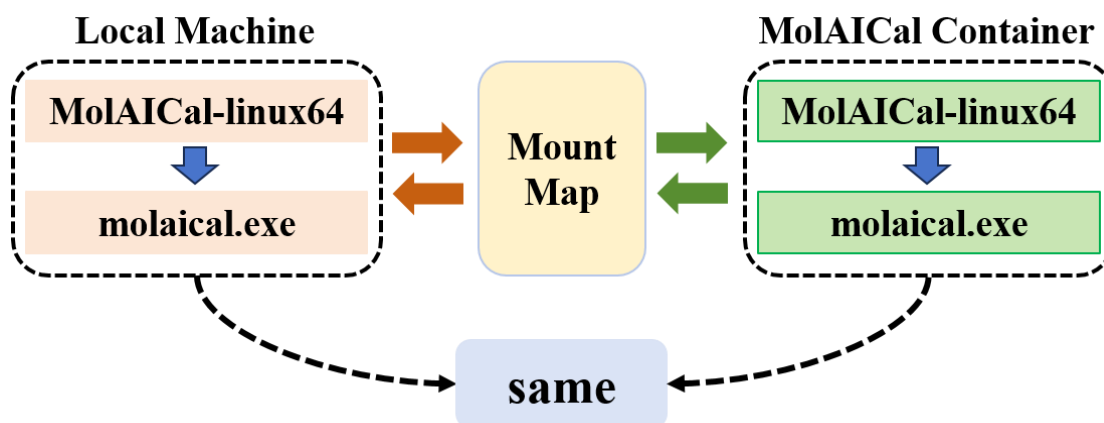


Figure 6.1 Architecture Diagram for the Linux Version of MolAICal

As shown in Figure 6.1, the **MolAICal container** can not only utilize the container system's files and programs but also access the host machine's library files through mounting and mapping mechanisms. This effectively **resolves** issues such as **library conflicts** or missing dependencies. However, **program execution within containers may indeed be slower than on the local machine.**

Figure 6.1 demonstrates that due to the mapping relationship, the computational commands executed by "**molaical.exe**" in the container are **identical** to those run by "**molaical.exe**" on the local machine.

- ◆ Therefore, to improve the running speed, **for tasks with minimal library dependencies**—such as virtual screening using MolAICal—**consider copying MolAICal from the container to the local machine:**

```

***** 1. Go to the container file system, it will enter the "/root" *****
#> molaical.exe -cset shell in

```

*** 2. The first part of 'cp' is from the MolAICal container, and the second part is in the local machine ***

```
#> cp -r soft/MolAICal-linux64 <local machine directory>/
```

***** 3. Exit container file system *****

```
#> exit
```

- ◆ At last, performing virtual screening locally via the same methodology can significantly accelerate the screening process.

Appendix 1

Configure Docker Desktop by clicking **Settings** → **Docker Engine** → "add content" → **Apply & restart**, then enter the repositories in the left-hand box (see Figure a1).

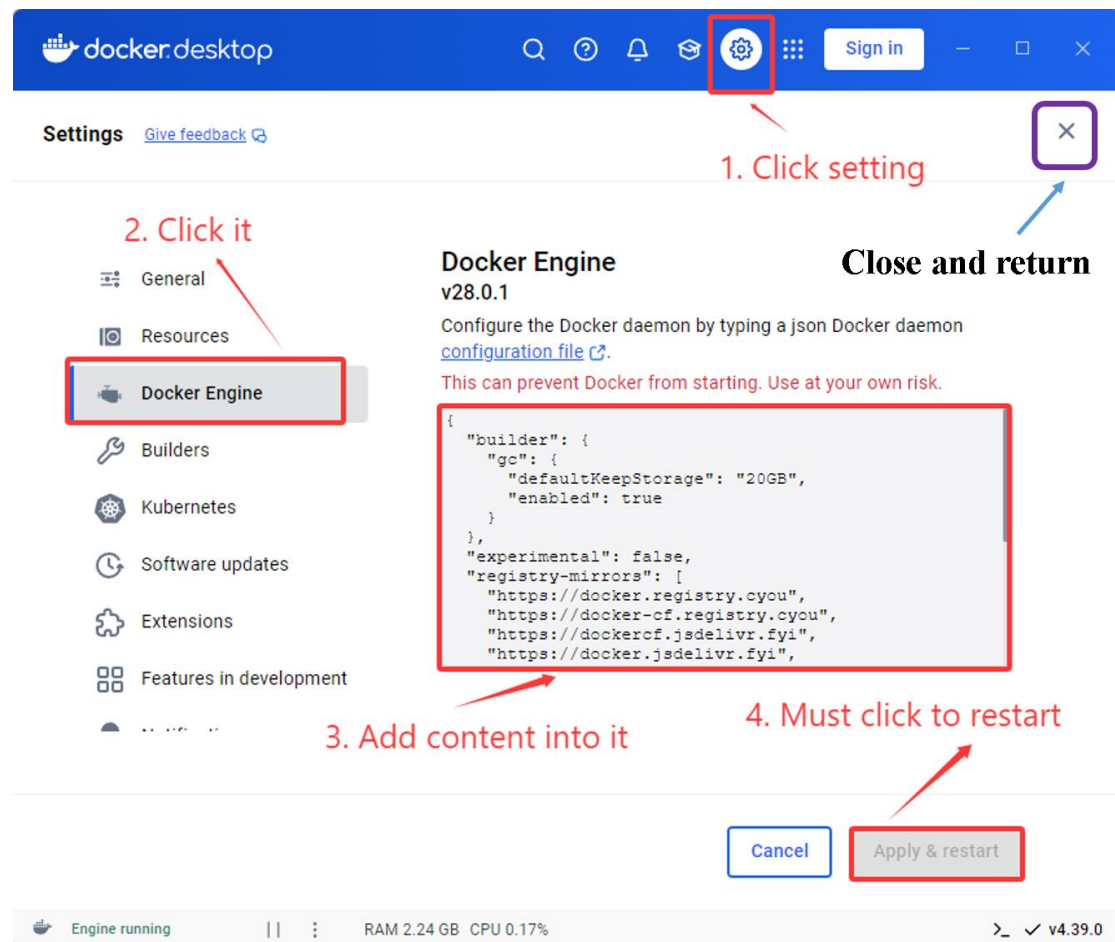


Figure a1

The configuration content for the image registry source is typically:

- ✧ Each item should be separated by a comma ",".
- ✧ The "registry-mirrors" field tells Docker Desktop about the new image registry source. The example provided here is for Chinese image sources and may not be guaranteed to work in other regions or countries. If not available, please replace it with a suitable registry source that you can connect to, following the format.

The enter content repositories are below:

```
{
  "builder": {
    "gc": {
      "defaultKeepStorage": "20GB",
```

```
    "enabled": true
  }
},
"experimental": false,
"registry-mirrors": [
  "https://docker.registry.cyou",
  "https://docker-cf.registry.cyou",
  "https://dockercf.jsdelivr.fyi",
  "https://docker.jsdelivr.fyi",
  "https://dockertest.jsdelivr.fyi",
  "https://mirror.aliyuncs.com",
  "https://dockerproxy.com",
  "https://mirror.baidubce.com",
  "https://docker.m.daocloud.io",
  "https://docker.nju.edu.cn",
  "https://docker.mirrors.sjtug.sjtu.edu.cn",
  "https://docker.mirrors.ustc.edu.cn",
  "https://mirror.iscas.ac.cn",
  "https://docker.rainbond.cc"
]
}
```